

AgentRecommender: LLM Agents Enable Customizable Recommender Systems on the User Side

Ryoma Sato

National Institute of Informatics

rsato@nii.ac.jp

Abstract

Recommender systems have traditionally been developed for platforms. However, this has given rise to many phenomena that may be advantageous for platform lock-in but are a nuisance to users, such as clickbait, filter bubbles, and the spread of fake news. Recently, user-side recommender systems have been proposed as a new paradigm for solving this problem. If users deploy their own recommender systems, they are no longer at the mercy of the platform’s interests. However, building a user-side recommender system is not trivial; in particular, customizing one for oneself requires additional data. We propose AgentRecommender, a method that leverages the investigation capability and internal knowledge of LLM agents to flexibly build user-side recommender systems without additional data. AgentRecommender allows users to easily create recommender systems tailored to their own preferences.

1 Introduction

Since their success at Amazon [24] and Netflix [10], recommender systems have been developed for platforms. To increase sales, they recommend products by predicting the probability that a user will purchase them; to keep users on the platform longer, or to make them watch more advertisements, they recommend content by predicting expected watch time. Recommender systems have brought platforms substantial profits. It is said that 35 percent of the sales at Amazon [30], 80 percent of the time consumed at Netflix [10], and more than 50 percent of the content on Instagram [6] are delivered through recommender systems.

It is true that recommender systems have helped platforms grow, and users have also often been helped by recommender systems in finding items they like. However, recommender systems have not always worked purely in users’ favor. A recommender system that optimizes the probability of purchase may recommend products that merely look good and do not serve the user, and in the hands of a recommender system that maximizes expected watch time, the user may end up being shown one video after another that keeps dragging out its conclusion. Recommender systems have also caused more serious problems, such as filter bubbles [16, 34, 50, 52] and unfairness [12, 19, 64]. Platform operators may fix these problems immediately when they hurt their own interests, for example when users start leaving, but otherwise they may not actively fix them. In such cases, there is little that users can do.

To solve this problem, Sato [40] proposed user-side recommender systems, which allow users to build recommender systems with the functionality they desire by themselves. For example, when the platform’s recommender system delivers news biased toward a particular political party, a user-side recommender system can balance the parties of the recommended items. When the platform’s recommender system delivers only popular items, a user-side recommender system can recommend minor yet relevant items. Unlike the platform, users are under severe constraints: they cannot directly access the database, nor can they obtain the log data of other users. Sato [40] solved this problem by cleverly using the recommendation network.

However, conventional user-side recommender systems [40, 42, 44] require item labels to implement the desired functionality. For example, when correcting for popularity, play counts or like counts can be used directly if they are public, as on video streaming services, but a movie streaming site may not disclose view counts. Likewise, some news sites may provide tags, but when they do not, telling which political party an

article favors is not trivial. One could resolve this by using an additional classifier, but preparing a classifier from scratch is a heavy burden for an ordinary user.

We solve this problem by using LLM agents. Since LLM agents possess world knowledge, they can often estimate item attributes zero-shot, and when necessary, they can estimate item attributes even more reliably by combining this with search and other tools. By combining them with user-side recommender systems, users can easily build recommender systems with the desired functionality. The proposed method, AgentRecommender, can realize the desired functionality on the user’s side even in regimes where conventional user-side recommender systems do not work.

2 Problem Setting

Following the setting of Sato [40], we assume that the service provider offers an item-to-item recommender system. That is, when the page of item $i \in \mathcal{I}$ is accessed, a set of related items $\mathcal{P}_{\text{Prov}}(i) \in \mathcal{I}^K$ is provided, where $\mathcal{I}$ is the set of all items. The service provider’s recommender system is not necessarily fair, nor does it necessarily have the properties the user desires. As such a system, we can exploit, for example, Amazon’s “Customers Who Bought This Item Also Bought” section or YouTube’s “Up next” section. We also assume that each item i has a hidden attribute $a_i \in \mathcal{A}$. Any kind of attribute is conceivable: for example, genres such as $\mathcal{A} = \{\text{“action”}, \text{“horror”}, \text{“comedy”}\}$, eras such as $\mathcal{A} = \{\text{“1990s”}, \text{“2000s”}, \text{“2010s”}, \text{“2020s”}\}$, popularity such as $\mathcal{A} = \{\text{“major”}, \text{“minor”}\}$, price ranges such as $\mathcal{A} = \{\text{“low-price”}, \text{“high-price”}\}$, and political leanings such as $\mathcal{A} = \{\text{“Democratic”}, \text{“Republican”}\}$. The user may design this attribute partition arbitrarily; we make no assumptions whatsoever about the categorization and leave it to the user. This is precisely the strength of realizing arbitrary functionality with user-side recommender systems. We assume that the user wants the recommended items to have these attributes in specific proportions, to be balanced, or to achieve higher coverage in the sense that at least one item from each category is recommended. The service provider’s recommender system does not necessarily respect these constraints. Under such circumstances, the user builds a recommender system that respects the desired constraints. A user who is tired of being recommended only famous items may specify popularity as the category so as to receive recommendations with balanced popularity, and a user who is fed up with recommendations biased toward a particular political leaning may specify political leaning as the category so as to receive politically balanced recommendations. The major difference from previous user-side recommender systems is that we assume the attribute $a_i \in \mathcal{A}$ is not observable to the user, in contrast to what previous user-side recommender systems assumed. This drastically reduces the amount of data the user needs to build a recommender system. The attributes $a_i \in \mathcal{A}$ are observed only at evaluation time, where we evaluate how well the recommender system built by the user satisfies the constraints.

In summary, the problem setting is stated as follows.

Given: Oracle access to the official recommendations $\mathcal{P}_{\text{Prov}}$; the attribute space $\mathcal{A}$ and the constraints to be satisfied, specified by the user in natural language; and a budget B . The attribute $a_i \in \mathcal{A}$ of each item is not observed at construction time and is referred to only at evaluation time.

Output: A user-side recommender system $\mathcal{Q}: \mathcal{I} \rightarrow \mathcal{I}^K$ that satisfies the constraints on $\mathcal{A}$. Internally, $\mathcal{Q}$ may evaluate $\mathcal{P}_{\text{Prov}}$ at most B times per recommendation.

For simplicity, we consider user-side item-to-item recommendation here, but user-side user-to-item recommendation can be constructed in the same manner.

3 Proposed Method: AgentRecommender

The proposed method, AgentRecommender, uses an LLM agent to build a recommendation list that satisfies the constraints on the item attributes $\mathcal{A}$ while using the service provider’s recommender system $\mathcal{P}_{\text{Prov}}$ (Figure 1). AgentRecommender repeatedly observes $\mathcal{P}_{\text{Prov}}(j)$ of an item j that the LLM deems promising and adds it to the candidates, thereby gradually collecting promising items, and in the end, the LLM constructs a list that satisfies the conditions from the candidates. This process can also be viewed as an LLM agent actively

config: **requirement**

user  visits item **i**

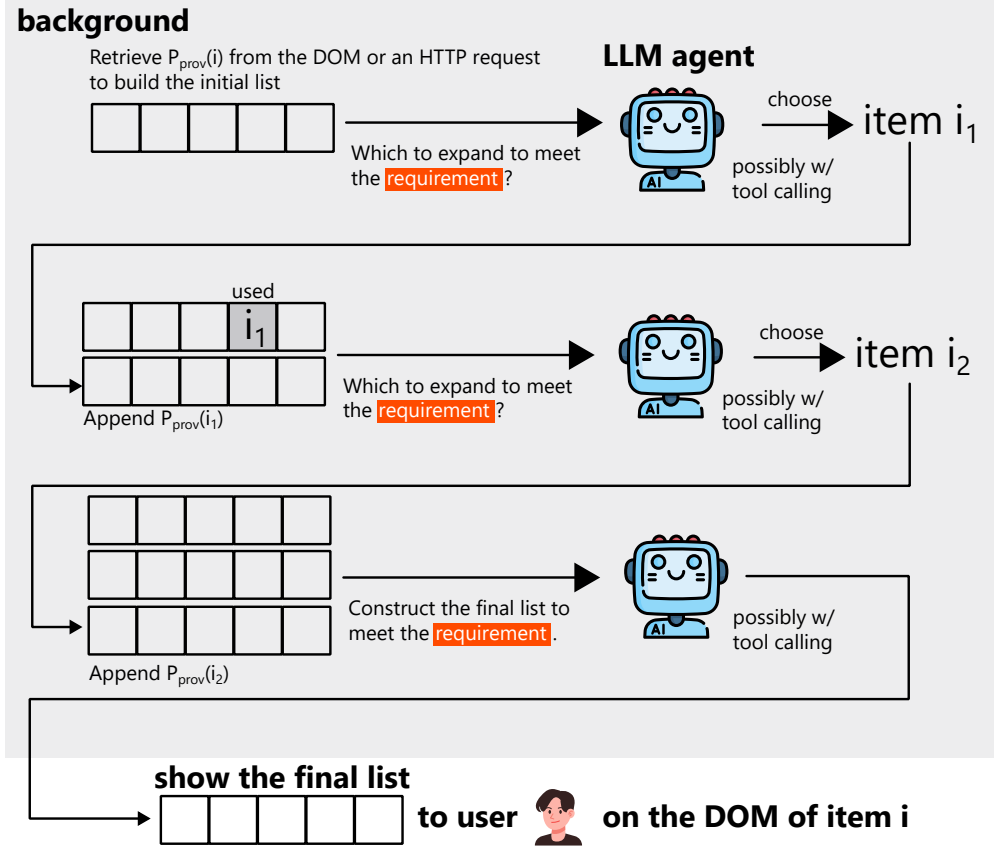


Figure 1: Conceptual illustration of AgentRecommender. AgentRecommender uses an LLM agent to build a recommendation list that satisfies the constraints while using the service provider’s recommender system $\mathcal{P}_{\text{prov}}$.

exploring the recommendation network, i.e., the graph whose nodes are the items and in which an edge is drawn from i to j when item j is recommended on the page of item i .

Suppose the user accesses the page of item i . The service provider’s recommendations $L = \mathcal{P}_{\text{prov}}(i) \in \mathcal{I}^K$ can then be observed from the DOM, HTTP requests, and the like. Starting from this list as the initial list, we repeat the following operations B times using the LLM:

1. Present to the LLM, in natural language, the items in the current list that have not yet been expanded together with the requirements, and let it select the item to expand.
2. Access the page of the selected item j behind the browser, obtain the service provider’s recommendations $\mathcal{P}_{\text{prov}}(j) \in \mathcal{I}^K$, and add them to the list L .

Finally, we present the items of L obtained so far and the requirements to the LLM, and let it select the K items that should ultimately be presented to the user.

Only the item names are given to the LLM, but the LLM estimates the attributes of the items using its internal knowledge (and, if necessary, tool calls such as search) and selects the item that should be explored

Table 1: **Experimental Results.** The proposed method, AgentRecommender, achieves far higher performance than the service provider’s recommender system and than merely re-ranking it, across the three domains of MovieLens 1M (movies), LastFM (music), and Amazon Home and Kitchens (e-commerce) and the three criteria of oldness, genre, and price. This shows that even when the service provider’s recommender system does not offer a particular functionality, users can build a recommender system with the desired functionality on their own.

	ML-1M success (oldness) $\uparrow$	ML-1M coverage (genre) $\uparrow$	LastFM coverage (genre) $\uparrow$	Amazon success (price) $\uparrow$
Provider	0.00 ± 0.00	5.51 ± 2.35	2.50 ± 1.02	0.16 ± 0.37
iAgent [58]	0.00 ± 0.00	5.51 ± 2.35	2.50 ± 1.02	0.16 ± 0.37
AgentRecommender	0.68 ± 0.47	8.94 ± 2.43	4.34 ± 1.62	0.64 ± 0.48

to satisfy the requirements. For example, if the current list is overly skewed toward action movies, the LLM would select a horror movie as the item j to expand. The recommendations for j are likely to contain horror movies, and since they are added to the list, the requirements become easier to satisfy. The list L ends up containing items different from those presented by the original recommender system, but since the included items are at most B hops away from the starting point, they are expected to be highly related to the original recommendations. Moreover, when we finally let the LLM select items from the list, the relevance issue can also be resolved by including in the prompt the requirement that the selection be restricted to items that are sufficiently related to the original item.

We have considered item-to-item recommendation here, but user-side user-to-item recommendation is also possible by using the service provider’s user-to-item recommendation results as the initial list L .

Issuing B additional queries every time the user accesses an item page may incur too much latency or place a burden on the service. In that case, as in RecCycle [44], we can exploit a cache of the recommendation results obtained during the user’s ordinary use of the service: instead of issuing queries online, we issue queries to a cache database built in, for example, the browser’s localstorage, thereby suppressing both the latency and the load on the service.

4 Experiments

We validate the effectiveness of AgentRecommender on three datasets: MovieLens 1M (movies) [13], LastFM (music) [5], and Amazon Home and Kitchens (e-commerce) [14, 29]. As the LLM, we use GPT-5.6 Luna (reasoning_effort=low) and receive its outputs as structured outputs with JSON schemas. In each expansion turn, we constrain the output with an enum listing the candidate ids and let the LLM select exactly one item; in the final selection, we let it select K distinct items from the accumulated candidate pool.

The experimental setup is as follows. First, we fix a single user. We perform item-to-item recommendation for the item this user interacted with last. The service provider’s recommender $\mathcal{P}_{\text{prov}}$ is implemented by obtaining item embeddings with Bayesian Personalized Ranking (BPR) [38] and recommending the items whose embeddings are close. Note that neither the embeddings nor the fact that the system is built with BPR can be exploited when building the user-side recommender system: at construction time, only black-box access to $\mathcal{P}_{\text{prov}}$ is allowed. As the requirements on the categories, we use one requiring that every category be included in the recommendation results at least τ times, for which we measure the rate at which it is achieved, and one requiring that the recommendation results include as many kinds of categories as possible, for which we measure the average number of unique categories. Throughout the following experiments, we set the number of recommendations to $K = 10$ and the budget to $B = 8$. We randomly select $n = 100$ users from the dataset and build a recommender system on the user’s side. We report the mean and standard deviation of the evaluation values across users.

4.1 MovieLens 1M

MovieLens 1M is a movie rating dataset containing 3,883 movies. We conduct experiments with the goal of being recommended movies of various genres and eras. First, using genre as the category, we evaluate the number of distinct genres included among the $K = 10$ recommended items; larger values indicate more diversity and are better. Note that a single movie can belong to multiple genres. Second, we set $a_i = \text{old}$ if the movie was released in 1989 or earlier and $a_i = \text{new}$ if it was released in 1990 or later, and we evaluate the rate at which both are included in the recommendation results with $\tau = 5$ items each. Note that the release year at the end of a movie title (e.g., “(1977)” in “Star Wars (1977)”) is removed from the titles presented to the agent. Therefore, the LLM cannot read the year directly from the title and must estimate oldness based on world knowledge. Larger values indicate a higher rate of balanced recommendations and are better.

The results are reported in the first and second columns of Table 1. The provider and iAgent do not succeed at all under the oldness criterion. This is because many recommender systems recommend only new items for new items and only old items for old items. However, this makes the recommendations biased, and users may be dissatisfied. Since iAgent merely reorders the items, it cannot improve the score over the provider’s recommender system. Even under such a difficult criterion, the proposed AgentRecommender produces recommendations with spread-out release years in as many as 68 percent of the cases. In the remaining 32 percent, it does not fail completely either: although the categories are not split half-and-half, the recommendations come close to it. Under the genre coverage criterion as well, AgentRecommender succeeds in including, on average, 1.6 times as many genres in the recommendation list as the provider and iAgent. These results show that AgentRecommender is beneficial for users who want diverse recommendations.

4.2 LastFM

LastFM is a dataset of music artists. We filter out artists with fewer than 15 users, leaving 1,052 artists after preprocessing. We conduct experiments with the goal of being recommended artists of various genres. The evaluation value is the number of distinct genres included among the $K = 10$ recommended items; larger values indicate more diversity and are better.

The results are reported in the third column of Table 1. Here as well, as with MovieLens, AgentRecommender succeeds in including, on average, 1.7 times as many genres in the recommendation list as the provider and iAgent. These results show that AgentRecommender is beneficial for users who want diverse recommendations.

4.3 Amazon Home and Kitchens

Amazon Home and Kitchens is a product review dataset. We start from the 5-core subset released by McAuley et al. (every user and every item has at least five reviews), and from it we keep only the products that have a title, have a price of at least \$1, and have a Home and Kitchen subcategory (the second level of the category path, such as Kitchen & Dining or Bedding). After this filtering, we keep only the users who have interacted with at least two of the remaining items. After preprocessing, 23,927 products remain. Among e-commerce recommender systems, one that recommends simply based on relatedness or co-purchases may recommend only low-priced products for low-priced products, and the user’s product search may hit a dead end; a recommender system that aims to maximize the platform’s profit may aggressively recommend only high-priced products, and the user may end up buying an expensive product without noticing the existence of cheaper products that would actually be sufficient. We conduct experiments with the goal of being recommended both low-priced and high-priced products. Specifically, we take the median price of the products in this category, \$19.99, as the threshold, and set $a_i = \text{low-price}$ for products below \$19.99 and $a_i = \text{high-price}$ for products at or above \$19.99. We evaluate the rate at which both are included in the recommendation results with $\tau = 5$ items each. Larger values indicate a higher rate of balanced recommendations and are better.

The results are reported in the fourth column of Table 1. Here as well, AgentRecommender achieves, on average, four times the success rate of the provider and iAgent. In other words, for users who want to

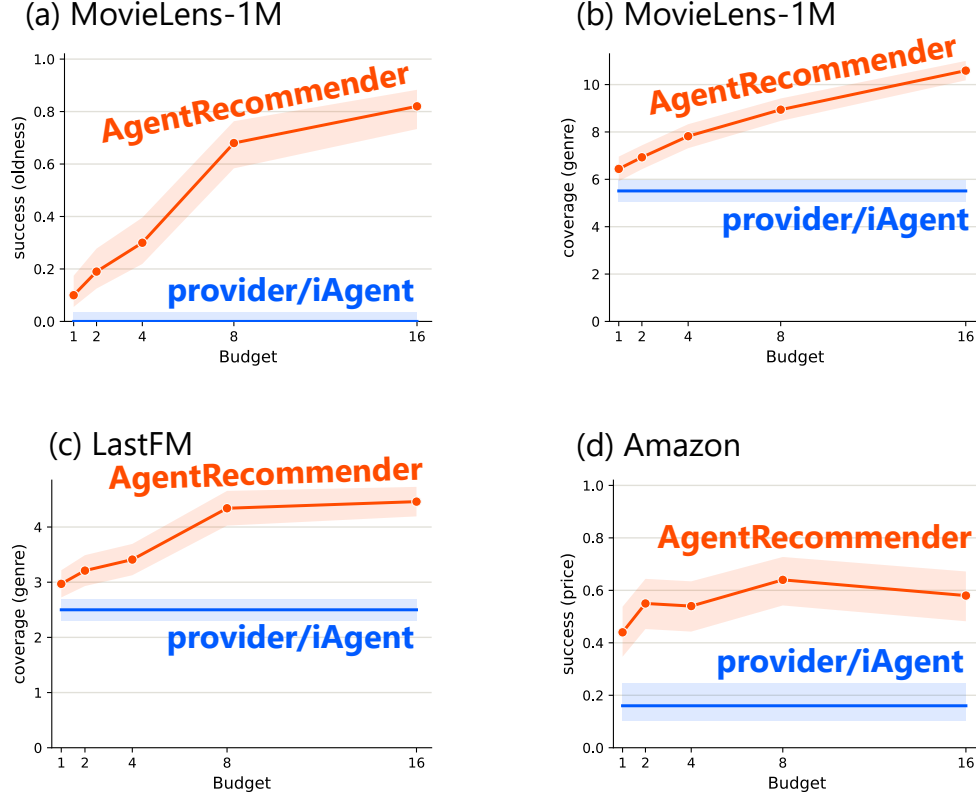


Figure 2: Budget variation experiment. AgentRecommender greatly outperforms the provider and iAgent even with a budget of $B = 1$, and it can improve its performance further as the budget increases. The bands represent 95 percent confidence intervals.

consider both cheap and expensive products, AgentRecommender can provide such opportunities even when the service provider’s official recommender system does not.

4.4 Budget Sensitivity

So far, we have fixed the budget to $B = 8$. Figure 2 shows the results when the budget is varied from $B = 1$ to $B = 16$. In all four settings, AgentRecommender already greatly outperforms the provider and iAgent at $B = 1$. This shows that AgentRecommender works even with a small budget and is practical. As the budget increases, the performance generally improves. The exception is the Amazon dataset; we attribute this to the fact that, on the Amazon dataset, estimating prices from product names alone is difficult, so even when a larger budget provides an abundant pool of candidate products, achieving a perfect price balance remains difficult, and the success rate plateaus around 0.6.

5 Related Work

User-side Realization. User-side realization refers to realizing desired functionality of a service on the user’s side [45]. Even a service that a user finds convenient and keeps using is bound to have some inconveniences. However, users have little power, and sending requests to the service does not guarantee that the inconvenience will be resolved. Since the data and content are locked in on the platform side, it is difficult for users to build a service from scratch by themselves. In this situation, user-side realization algorithms take a middle path: they realize the functionality the user desires while still using the service. Representative examples include user-side search engines [39, 41], user-side privacy protection [43], and user-side watermarking of large language models [47].

Most closely related to this work are user-side recommender systems. PrivateWalk and PrivateRank are the first user-side recommender systems [40]. However, they are not practical because they require a large number of accesses to the official recommender system or degrade the performance. Consul [42] is the first practical user-side recommender system with multiple desirable properties. RecCycle [44] improved the efficiency further and realized an effective user-side recommender system with no overhead. However, all of these require item labels, which becomes a burden on the user when the service does not provide them. iAgent [58] is a recently proposed user-side recommender system that uses an agent. However, it merely reorders the items presented by the service, and it essentially cannot discover new items. The proposed method is the first user-side recommender system that requires no labels and can discover new, useful items. As a closely related idea, Bonsai [27] personalizes social feeds by describing the user’s preferences in natural language and selecting and ranking posts while performing search and other operations. Alexandria [21] and Value Alignment [17] are also user-side methods for personalizing social feeds: when the user specifies preferred attributes, an LLM predicts the attributes of the feed and reorders the feed items to match the user’s preferences. These can also be regarded as a kind of user-side realization. Stray et al. [49] conducted a large-scale examination of the effects of such user-side re-ranking of social feeds. Yuan et al. [63] and Lazar et al. [23] argue for making recommender systems agent-centric, partly for the sake of user-side controllability. This argument is also made in Sato [45]. In a parallel direction, there is also a method that transfers recommendation data locked into one service to another service on the user’s side [46], which can be used as a direction orthogonal to the present method.

Separately from these lines of work, methods that incorporate LLM agents into recommender systems have also been proposed [15, 20, 22, 28, 37, 54, 55, 59, 60, 62, 66]. However, these are basically assumed to run on the service provider’s servers, not on the user’s side. For example, TEARS [35] enables users to convey their preferences to the recommender system in natural language, but it runs on the service provider’s servers, so users cannot use it unless the service provider implements TEARS. The advantage of user-side realization is that it can be achieved by the user’s own power alone, without the provider implementing the method or offering endpoints. In addition, methods that simulate users with LLMs, such as AgentCF [25, 67] and RecAgent [53], have been proposed, but they are mainly used for modeling and evaluation on the provider side.

Fairness in Recommender Systems. As fairness has become a major concern in society [33, 36], many fairness-aware machine learning algorithms have been proposed [12, 19, 64]. In particular, fairness with respect to gender [48, 57, 65], race [57, 65], financial status [8], and popularity [31, 56] is of great concern. In light of this, many fairness-aware recommendation algorithms have been proposed [3, 18, 32, 61]. Some of them aim to ensure fairness for users [4] and others aim to ensure fairness for items (such as products and accounts) [2, 7, 9, 26, 31], and some aim to ensure fairness for both users and items [4]. In this study, we focus on fairness for items following [40, 42], but the proposed method can also be applied to fairness for users when the official system is not fair for the very user that uses the user-side system. The user can overcome the unfairness of the official system by creating their own recommender system. Note that fairness is closely related to topic diversification [68] by regarding the topic as the sensitive attribute, and we considered the diversity of recommended items in this study as well.

Steerable Recommender Systems. The reliability of recommender systems has attracted a lot of attention [1, 51], and steerable recommender systems that let the users modify the behavior of the system have been proposed [1, 11]. User-side recommender systems also allow the users to modify the recommendation results. However, the crucial difference between steerable and user-side recommender systems is that steerable recommender systems must be implemented by a service provider, whereas user-side recommender systems can be built by arbitrary users even if the official system is an ordinary (non-steerable) one. Therefore, user-side recommender systems can expand the scope of steerable recommender systems by a considerable margin [40].

6 Conclusion

We proposed AgentRecommender, a method for building a recommender system with desired functionality on the user’s side, and moreover without additional labels. AgentRecommender actively explores items that satisfy the desired requirements while using the service provider’s recommender system, and presents them to the user. AgentRecommender can realize the desired functionality within the scope of the user’s privileges, skillfully steering clear of the areas an ordinary user cannot access, such as the database and the log data of other users. Moreover, by exploiting the internal knowledge and search capability of LLM agents, it avoids burdensome work such as labeling. This makes recommender systems much easier for ordinary users to build. In our experiments, across the three domains of movies, music, and e-commerce, we succeeded in creating recommender systems that satisfy the requirements even though the service providers’ recommender systems did not satisfy them.

References

- [1] K. Balog, F. Radlinski, and S. Arakelyan. Transparent, scrutable and explainable user models for personalized recommendation. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR*, pages 265–274, 2019. doi: 10.1145/3331184.3331211. URL <https://doi.org/10.1145/3331184.3331211>.
- [2] A. Beutel, J. Chen, T. Doshi, H. Qian, L. Wei, Y. Wu, L. Heldt, Z. Zhao, L. Hong, E. H. Chi, and C. Goodrow. Fairness in recommendation ranking through pairwise comparisons. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD*, pages 2212–2220, 2019. doi: 10.1145/3292500.3330745. URL <https://doi.org/10.1145/3292500.3330745>.
- [3] A. J. Biega, K. P. Gummadi, and G. Weikum. Equity of attention: Amortizing individual fairness in rankings. In *Proceedings of the 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, SIGIR*, pages 405–414, 2018. doi: 10.1145/3209978.3210063. URL <https://doi.org/10.1145/3209978.3210063>.
- [4] R. Burke. Multisided fairness for recommendation. In *Proceedings of the 4th Workshop on Fairness, Accountability, and Transparency in Machine Learning, FAT/ML*, 2017.
- [5] I. Cantador, P. Brusilovsky, and T. Kuflik. Second workshop on information heterogeneity and fusion in recommender systems (HetRec2011). In *Proceedings of the 5th ACM Conference on Recommender Systems, RecSys*, pages 387–388, 2011. doi: 10.1145/2043932.2044016. URL <https://doi.org/10.1145/2043932.2044016>.
- [6] J. Deng. Meta now has an AI chatbot. experts say get ready for more AI-powered social media, 2024. URL <https://www.latimes.com/world-nation/story/2024-05-02/ai-chatbots-come-to-meta-more-social-media>. Accessed: 2026-03-04.
- [7] M. D. Ekstrand, M. Tian, M. R. I. Kazi, H. Mehrpouyan, and D. Kluver. Exploring author gender in book rating and recommendation. In *Proceedings of the 12th ACM Conference on Recommender Systems, RecSys*, pages 242–250, 2018. doi: 10.1145/3240323.3240373. URL <https://doi.org/10.1145/3240323.3240373>.
- [8] Z. Fu, Y. Xian, R. Gao, J. Zhao, Q. Huang, Y. Ge, S. Xu, S. Geng, C. Shah, Y. Zhang, and G. de Melo. Fairness-aware explainable recommendation over knowledge graphs. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR*, pages 69–78, 2020. doi: 10.1145/3397271.3401051. URL <https://doi.org/10.1145/3397271.3401051>.
- [9] S. C. Geyik and K. Kenthapadi. Building representative talent search at LinkedIn, 2018. URL <https://engineering.linkedin.com/blog/2018/10/building-representative-talent-search-at-linkedin>.
- [10] C. A. Gomez-Uribe and N. Hunt. The Netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems*, 6(4):13:1–13:19, 2015. doi: 10.1145/2843948. URL <https://doi.org/10.1145/2843948>.

-
- [11] S. J. Green, P. Lamere, J. Alexander, F. Maillet, S. Kirk, J. Holt, J. Bourque, and X. Mak. Generating transparent, steerable recommendations from textual descriptions of items. In *Proceedings of the 3rd ACM Conference on Recommender Systems, RecSys*, pages 281–284, 2009. doi: 10.1145/1639714.1639768. URL <https://doi.org/10.1145/1639714.1639768>.
 - [12] M. Hardt, E. Price, and N. Srebro. Equality of opportunity in supervised learning. In *Advances in Neural Information Processing Systems, NIPS*, volume 29, pages 3315–3323, 2016.
 - [13] F. M. Harper and J. A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015. doi: 10.1145/2827872. URL <https://doi.org/10.1145/2827872>.
 - [14] R. He and J. J. McAuley. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *Proceedings of the 25th International Conference on World Wide Web, WWW*, pages 507–517, 2016. doi: 10.1145/2872427.2883037. URL <https://doi.org/10.1145/2872427.2883037>.
 - [15] J. Huang, X. Zou, L. Xia, and Q. Li. MR.Rec: Synergizing memory and reasoning for personalized recommendation assistant with LLMs. *arXiv preprint arXiv:2510.14629*, 2025. doi: 10.48550/arXiv.2510.14629. URL <https://arxiv.org/abs/2510.14629>.
 - [16] E. Hussein, P. Juneja, and T. Mitra. Measuring misinformation in video search platforms: An audit study on YouTube. *Proceedings of the ACM on Human-Computer Interaction*, 4(CSCW1):1–27, 2020. doi: 10.1145/3392854. URL <https://doi.org/10.1145/3392854>.
 - [17] F. Jahanbakhsh, D. Zhao, T. Piccardi, Z. Robertson, Z. Epstein, S. Koyejo, and M. S. Bernstein. Value alignment of social media ranking algorithms. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, CHI*, pages 1401:1–1401:26, 2026. doi: 10.1145/3772318.3791281. URL <https://doi.org/10.1145/3772318.3791281>.
 - [18] T. Kamishima, S. Akaho, H. Asoh, and J. Sakuma. Enhancement of the neutrality in recommendation. In *Proceedings of the 2nd Workshop on Human Decision Making in Recommender Systems, Decisions@RecSys*, volume 893, pages 8–14, 2012.
 - [19] T. Kamishima, S. Akaho, H. Asoh, and J. Sakuma. Fairness-aware classifier with prejudice remover regularizer. In *Machine Learning and Knowledge Discovery in Databases, ECML PKDD*, volume 7524, pages 35–50, 2012. doi: 10.1007/978-3-642-33486-3_3. URL https://doi.org/10.1007/978-3-642-33486-3_3.
 - [20] T. Khezresmaeilzadeh, J. Zhang, D. Andreadis, and K. Psounis. Preserving privacy and utility in LLM-based product recommendations. *arXiv preprint arXiv:2505.00951*, 2025. doi: 10.48550/arXiv.2505.00951. URL <https://arxiv.org/abs/2505.00951>.
 - [21] A. Kolluri, R. Su, F. Jahanbakhsh, D. Zhao, T. Piccardi, and M. S. Bernstein. Alexandria: A library of pluralistic values for realtime re-ranking of social media feeds. *Proceedings of the International AAAI Conference on Web and Social Media*, 20(1):1309–1330, 2026. doi: 10.1609/icwsm.v20i1.42697. URL <https://doi.org/10.1609/icwsm.v20i1.42697>.
 - [22] X. Kong, J. Jiang, B. Liu, Z. Xu, H. Zhu, J. Xu, B. Zheng, J. Wu, and X. Wang. Think before recommendation: Autonomous reasoning-enhanced recommender. In *Advances in Neural Information Processing Systems, NeurIPS*, volume 38, pages 156622–156645, 2025. doi: 10.52202/085713-4718. URL <https://doi.org/10.52202/085713-4718>.
 - [23] S. Lazar, L. Thorburn, T. Jin, and L. Belli. The moral case for using language model agents for recommendation. *Inquiry*, pages 1–25, 2025. doi: 10.1080/0020174x.2025.2515579. URL <https://doi.org/10.1080/0020174x.2025.2515579>.
 - [24] G. Linden, B. Smith, and J. York. Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Computing*, 7(1):76–80, 2003. doi: 10.1109/mic.2003.1167344. URL <https://doi.org/10.1109/mic.2003.1167344>.
 - [25] J. Liu, S. Gu, D. Li, G. Zhang, M. Han, H. Gu, P. Zhang, T. Lu, L. Shang, and N. Gu. AgentCF++: Memory-enhanced LLM-based agents for popularity-aware cross-domain recommendations. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Develop-*

-
- ment in Information Retrieval, *SIGIR*, pages 2566–2571, 2025. doi: 10.1145/3726302.3730161. URL <https://doi.org/10.1145/3726302.3730161>.
- [26] W. Liu, J. Guo, N. Sonboli, R. Burke, and S. Zhang. Personalized fairness-aware re-ranking for micro-lending. In *Proceedings of the 13th ACM Conference on Recommender Systems, RecSys*, pages 467–471, 2019. doi: 10.1145/3298689.3347016. URL <https://doi.org/10.1145/3298689.3347016>.
 - [27] O. E. Malki, M. A. L. Quéré, A. Monroy-Hernández, and M. H. Ribeiro. Bonsai: Intentional and personalized social media feeds. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, CHI*, pages 1395:1–1395:24, 2026. doi: 10.1145/3772318.3791855. URL <https://doi.org/10.1145/3772318.3791855>.
 - [28] R. Y. Maragheh, P. Vadla, P. Gupta, K. Zhao, A. Inan, K. Yao, J. Xu, P. Kanumala, J. H. D. Cho, and S. Kumar. ARAG: Agentic retrieval augmented generation for personalized recommendation. *arXiv preprint arXiv:2506.21931*, 2025. doi: 10.48550/arXiv.2506.21931. URL <https://arxiv.org/abs/2506.21931>.
 - [29] J. J. McAuley, C. Targett, Q. Shi, and A. van den Hengel. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR*, pages 43–52, 2015. doi: 10.1145/2766462.2767755. URL <https://doi.org/10.1145/2766462.2767755>.
 - [30] McKinsey & Company. How retailers can keep up with consumers, 2013. URL <https://www.mckinsey.com/industries/retail/our-insights/how-retailers-can-keep-up-with-consumers>. Accessed: 2026-03-04.
 - [31] R. Mehrotra, J. McInerney, H. Bouchard, M. Lalmas, and F. Diaz. Towards a fair marketplace: Counterfactual evaluation of the trade-off between relevance, fairness & satisfaction in recommendation systems. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management, CIKM*, pages 2243–2251, 2018. doi: 10.1145/3269206.3272027. URL <https://doi.org/10.1145/3269206.3272027>.
 - [32] S. Milano, M. Taddeo, and L. Floridi. Recommender systems and their ethical challenges. *AI & SOCIETY*, 35(4):957–967, 2020. doi: 10.1007/s00146-020-00950-y. URL <https://doi.org/10.1007/s00146-020-00950-y>.
 - [33] C. Muñoz, M. Smith, and D. Patil. Big data: A report on algorithmic systems, opportunity, and civil rights. Technical report, Executive Office of the President, 2016.
 - [34] E. Pariser. *The filter bubble: What the Internet is hiding from you*. Penguin UK, 2011.
 - [35] E. Penaloza, O. Gouvert, H. Wu, and L. Charlin. TEARS: Text representations for scrutable recommendations. In *Proceedings of the ACM on Web Conference 2025, WWW*, pages 4949–4968, 2025. doi: 10.1145/3696410.3714948. URL <https://doi.org/10.1145/3696410.3714948>.
 - [36] J. Podesta, P. Pritzker, E. J. Moniz, J. Holdren, and J. Zients. Big data: Seizing opportunities, preserving values. Technical report, White House, Executive Office of the President, 2014.
 - [37] S. Raj, A. V. Jha, M. Anand, and S. Saha. HARPO: Hierarchical agentic reasoning for user-aligned conversational recommendation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL*, pages 35580–35599, 2026. doi: 10.18653/v1/2026.acl-long.1646. URL <https://doi.org/10.18653/v1/2026.acl-long.1646>.
 - [38] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence, UAI*, pages 452–461, 2009.
 - [39] R. Sato. CLEAR: A fully user-side image search system. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, CIKM*, pages 4970–4974, 2022. doi: 10.1145/3511808.3557172. URL <https://doi.org/10.1145/3511808.3557172>.
 - [40] R. Sato. Private recommender systems: How can users build their own fair recommender systems without log data? In *Proceedings of the 2022 SIAM International Conference on Data Mining, SDM*, pages 549–557, 2022. doi: 10.1137/1.9781611977172.62. URL <https://doi.org/10.1137/1.9781611977172.62>.

-
- [41] R. Sato. Retrieving black-box optimal images from external databases. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining, WSDM*, pages 879–887, 2022. doi: 10.1145/3488560.3498462. URL <https://doi.org/10.1145/3488560.3498462>.
 - [42] R. Sato. Towards principled user-side recommender systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, CIKM*, pages 1757–1766, 2022. doi: 10.1145/3511808.3557476. URL <https://doi.org/10.1145/3511808.3557476>.
 - [43] R. Sato. Making translators privacy-aware on the user’s side. *Transactions on Machine Learning Research*, 2024. URL <https://arxiv.org/abs/2312.04068>.
 - [44] R. Sato. Overhead-free user-side recommender systems. *arXiv preprint arXiv:2411.07589*, 2024. doi: 10.48550/arXiv.2411.07589. URL <https://arxiv.org/abs/2411.07589>.
 - [45] R. Sato. *User-Side Realization*. PhD thesis, Kyoto University, 2024. URL <https://arxiv.org/abs/2403.15757>.
 - [46] R. Sato. Solving the cold start problem on one’s own as an end user via preference transfer. *Transactions on Machine Learning Research, TMLR*, 2025. doi: 10.48550/arxiv.2502.12398. URL <https://arxiv.org/abs/2502.12398>.
 - [47] R. Sato, Y. Takezawa, H. Bao, K. Niwa, and M. Yamada. Embarrassingly simple text watermarks. *arXiv preprint arXiv:2310.08920*, 2023. doi: 10.48550/arXiv.2310.08920. URL <https://arxiv.org/abs/2310.08920>.
 - [48] A. Singh and T. Joachims. Fairness of exposure in rankings. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD*, pages 2219–2228, 2018. doi: 10.1145/3219819.3220088. URL <https://doi.org/10.1145/3219819.3220088>.
 - [49] J. Stray, I. Baker, G. Beknazar-Yuzbashev, C. Budak, J. Kamin, K. Rutherford, M. Stalinski, T. Acosta, C. Bailey, M. S. Bernstein, M. Brandt, A. Bruckman, A. Chhabra, S. De, K. Duskin, S. Fish, B. Goldberg, A. Guess, D. Hadfield-Menell, M. Haroon, S. Hossain, M. Inzlicht, G. Jain, Y. Jiang, A. P. Landry, Y. Lelkes, H. Lu, P. Mason, J. McCoy, S. Milli, P. Resnick, E. Saltz, M. Saveski, L. Schirch, M. Spohn, S. Srinivasan, A. Tatore, L. Thorburn, J. A. Tucker, R. Willer, M. Wojcieszak, M. Wüthrich, and S. Zheng. The prosocial ranking challenge: Reducing polarization on social media without sacrificing engagement. *arXiv preprint arXiv:2603.19626*, 2026. doi: 10.48550/arXiv.2603.19626. URL <https://arxiv.org/abs/2603.19626>.
 - [50] The YouTube Team. Managing harmful conspiracy theories on YouTube, 2020. URL <https://blog.youtube/news-and-events/harmful-conspiracy-theories-youtube/>. Accessed: 2025-10-13.
 - [51] N. Tintarev and J. Masthoff. A survey of explanations in recommender systems. In *Proceedings of the 23rd IEEE International Conference on Data Engineering Workshop, ICDEW*, pages 801–810, 2007. doi: 10.1109/icdew.2007.4401070. URL <https://doi.org/10.1109/icdew.2007.4401070>.
 - [52] M. Tomlein, B. Pecher, J. Simko, I. Srba, R. Móro, E. Stefancova, M. Kompan, A. Hrkova, J. Podrouzek, and M. Bielíková. An audit of misinformation filter bubbles on YouTube: Bubble bursting and recent behavior changes. In *Proceedings of the 15th ACM Conference on Recommender Systems, RecSys*, pages 1–11, 2021. doi: 10.1145/3460231.3474241. URL <https://doi.org/10.1145/3460231.3474241>.
 - [53] L. Wang, J. Zhang, H. Yang, Z.-Y. Chen, J. Tang, Z. Zhang, X. Chen, Y. Lin, H. Sun, R. Song, W. X. Zhao, J. Xu, Z. Dou, J. Wang, and J.-R. Wen. User behavior simulation with large language model-based agents. *ACM Transactions on Information Systems*, 43(2):1–37, 2025. doi: 10.1145/3708985. URL <https://doi.org/10.1145/3708985>.
 - [54] M. Wang, C. Gao, W. Wang, Y. Li, and F. Feng. Tunable LLM-based proactive recommendation agent. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL*, pages 19262–19276, 2025. doi: 10.18653/v1/2025.acl-long.944. URL <https://doi.org/10.18653/v1/2025.acl-long.944>.
 - [55] S. Wozniak, J. Dusencko, J. Kocon, and P. Kazienko. Improving LLM-based recommender systems with user-controllable profiles. In *Companion Proceedings of the ACM on Web Conference 2025, WWW Companion*, pages 2102–2111, 2025. doi: 10.1145/3701716.3717734. URL <https://doi.org/10.1145/3701716.3717734>.

-
- [56] W. Xiao, H. Zhao, H. Pan, Y. Song, V. W. Zheng, and Q. Yang. Beyond personalization: Social content recommendation for creator equality and consumer satisfaction. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD*, pages 235–245, 2019. doi: 10.1145/3292500.3330965. URL <https://doi.org/10.1145/3292500.3330965>.
 - [57] R. Xu, P. Cui, K. Kuang, B. Li, L. Zhou, Z. Shen, and W. Cui. Algorithmic decision making with conditional fairness. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD*, pages 2125–2135, 2020. doi: 10.1145/3394486.3403263. URL <https://doi.org/10.1145/3394486.3403263>.
 - [58] W. Xu, Y. Shi, Z. Liang, X. Ning, K. Mei, K. Wang, X. Zhu, M. Xu, and Y. Zhang. iAgent: LLM agent as a shield between user and recommender systems. In *Findings of the Association for Computational Linguistics: ACL 2025, Findings of ACL*, pages 18056–18084, 2025. doi: 10.18653/v1/2025.findings-acl.928. URL <https://doi.org/10.18653/v1/2025.findings-acl.928>.
 - [59] M. Yang, N. Choudhary, J. Du, E. W. Huang, P. S. Yu, K. Subbian, and D. Koutra. AgentDR: Dynamic recommendation with implicit item-item relations via LLM-based agents. In *Proceedings of the ACM Web Conference 2026, WWW*, pages 6230–6241, 2026. doi: 10.1145/3774904.3792304. URL <https://doi.org/10.1145/3774904.3792304>.
 - [60] M. Yang, Z. Liu, W. Zhang, Y. Wang, H. Peng, and P. S. Yu. Personalized recommendation tool learning via autonomous language agents. In *Proceedings of the 20th ACM Conference on Recommender Systems, RecSys*, pages 722–727, 2026. doi: 10.1145/3773078.3831794. URL <https://doi.org/10.1145/3773078.3831794>.
 - [61] S. Yao and B. Huang. Beyond parity: Fairness objectives for collaborative filtering. In *Advances in Neural Information Processing Systems, NIPS*, volume 30, pages 2921–2930, 2017.
 - [62] H. Yu, Y. Wu, H. Wang, W. Guo, Y. Liu, Y. Li, Y. Ye, J. Du, and E. Chen. Thought-augmented planning for LLM-powered interactive recommender agent. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1, KDD*, pages 1821–1832, 2026. doi: 10.1145/3770854.3780286. URL <https://doi.org/10.1145/3770854.3780286>.
 - [63] H. Yuan, P. He, D. Zhang, J. Liang, and J. Zhu. Position: Recommender systems should move beyond platform-centric ranking toward personal agent-mediated recommendation. *arXiv preprint arXiv:2609.11942*, 2026. doi: 10.48550/arXiv.2609.11942. URL <https://arxiv.org/abs/2609.11942>.
 - [64] M. B. Zafar, I. Valera, M. Gomez-Rodriguez, and K. P. Gummadi. Fairness beyond disparate treatment & disparate impact: Learning classification without disparate mistreatment. In *Proceedings of the 26th International Conference on World Wide Web, WWW*, pages 1171–1180, 2017. doi: 10.1145/3038912.3052660. URL <https://doi.org/10.1145/3038912.3052660>.
 - [65] M. Zehlike, F. Bonchi, C. Castillo, S. Hajian, M. Megahed, and R. Baeza-Yates. FA*IR: A fair top-k ranking algorithm. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, CIKM*, pages 1569–1578, 2017. doi: 10.1145/3132847.3132938. URL <https://doi.org/10.1145/3132847.3132938>.
 - [66] H. Zhang, Y. Zhu, K. Mao, T. Li, and Z. Dou. RecThinker: An agentic framework for tool-augmented reasoning in recommendation. *arXiv preprint arXiv:2603.09843*, 2026. doi: 10.48550/arXiv.2603.09843. URL <https://arxiv.org/abs/2603.09843>.
 - [67] J. Zhang, Y. Hou, R. Xie, W. Sun, J. J. McAuley, W. X. Zhao, L. Lin, and J.-R. Wen. AgentCF: Collaborative learning with autonomous language agents for recommender systems. In *Proceedings of the ACM Web Conference 2024, WWW*, pages 3679–3689, 2024. doi: 10.1145/3589334.3645537. URL <https://doi.org/10.1145/3589334.3645537>.
 - [68] C. Ziegler, S. M. McNee, J. A. Konstan, and G. Lausen. Improving recommendation lists through topic diversification. In *Proceedings of the 14th International Conference on World Wide Web, WWW*, pages 22–32, 2005. doi: 10.1145/1060745.1060754. URL <https://doi.org/10.1145/1060745.1060754>.